
NATIONAL ECONOMICS UNIVERSITY (NEU)
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

OFFICIAL SOLUTIONS & MARKING RUBRIC

Course: Fundamental Programming Concepts in Python (DSAI1003)

Progress Exam: Decision Making and Boolean Logic in Python

Note for Instructors and Teaching Assistants: This document provides complete answer keys, rationale for conceptual questions, exact output traces, step-by-step program logic, reference Python implementations, and an itemized marking rubric designed for fair and consistent grading across all cohorts.

Section A: Decision Making & Boolean Logic Concepts (20 points)

2 points per question. Total = 20 points.

Q#	Key	Conceptual Explanation & Rule Reference	Marks
Q1	B	In Python, block structure is determined solely by consistent indentation. Mixing tabs and spaces or varying indentation within a block raises an <code>IndentationError</code> . Colons are mandatory, braces are not used for block delimiters, and <code>else</code> cannot exist standalone.	2 pts
Q2	C	The single equals sign <code>=</code> is the assignment operator. Python disallows assignment inside condition headers, raising a compile-time <code>SyntaxError: invalid syntax</code> . Equality must use <code>==</code> .	2 pts
Q3	B	Binary representation causes inexact precision for many decimal fractions ($0.1 + 0.2 \approx 0.30000000000000004$). Direct equality comparison <code>x == 0.3</code> evaluates to <code>False</code> . The standard numerical method is checking proximity: <code>abs(x - target) < EPSILON</code> .	2 pts
Q4	B	String comparison in Python is lexicographical, based on Unicode code points. Uppercase letters have lower numerical values than lowercase letters (ASCII/Unicode: 'B' is 66, 'a' is 97). Thus, <code>"apple" > "Banana"</code> is <code>True</code> , making <code>"apple" < "Banana"</code> evaluate to <code>False</code> .	2 pts
Q5	B	Python evaluates <code>if-elif</code> ladders sequentially from top to bottom. As soon as a condition evaluates to <code>True</code> , its block executes and all subsequent <code>elif</code> and <code>else</code> branches are bypassed. Since $85 \geq 60$, <code>grade = "D"</code> executes immediately.	2 pts
Q6	A	Operator precedence for boolean operators is: <code>not</code> (highest), followed by <code>and</code> , and finally <code>or</code> (lowest). Evaluation: <code>not False</code> $\rightarrow$ <code>True</code> ; <code>True and False</code> $\rightarrow$ <code>False</code> ; <code>False or True</code> $\rightarrow$ <code>True</code> .	2 pts
Q7	B	Python uses short-circuit evaluation . In an <code>and</code> expression, if the left operand is <code>False</code> , the overall expression cannot be true, so Python stops and returns <code>False</code> without evaluating the right operand, safely avoiding <code>ZeroDivisionError</code> .	2 pts
Q8	B	De Morgan's Law states: $\neg(A \vee B) \equiv (\neg A) \wedge (\neg B)$. Negating <code>status == "VIP"</code> gives <code>status != "VIP"</code> ; negating <code>points >= 100</code> gives <code>points < 100</code> ; the <code>or</code> operator flips to <code>and</code> .	2 pts
Q9	C	The built-in string method <code>s.isdigit()</code> returns <code>True</code> if all characters in the string are digits and there is at least one character; otherwise it returns <code>False</code> .	2 pts
Q10	C	Full branch coverage requires executing every possible control flow path. An <code>if-elif-elif-else</code> construct has four distinct executable branches; thus, at least four test cases are required (one triggering each branch).	2 pts

Section B: Output Tracing, Logic Diagnosis & Error Correction (25 points)

Q11. Exact Output Tracing with Nested Decisions & Surcharges (5 points)**Solution for Q11**

```
Base: $40.00 | Discount: 30%  
Payable: $28.00  
Eligible: True
```

Scoring Breakdown:

- **Line 1 (2 points):** Base: \$40.00 | Discount: 30% ($age = 19 < 25, is_student = True \rightarrow discount = 0.30$).
- **Line 2 (1.5 points):** Payable: \$28.00 ($final_fare = 40.0 \times (1.0 - 0.30) = 40.0 \times 0.70 = 28.0$, formatted as \$28.00).
- **Line 3 (1.5 points):** Eligible: True ($0.30 \geq 0.30$ is True and $19 \geq 18$ is True $\rightarrow$ True and True evaluates to True).

Q12. Evaluation of Boolean Expressions and Short-Circuiting (5 points)**Solution for Q12**

- **a) $10 < 20 \leq 25$: True** (1 pt) – Chained comparison equivalent to $(10 < 20)$ and $(20 \leq 25)$.
- **b) `"py" in "Python".lower()` and `not (5 > 8)`: True** (1 pt) – `"py" in "python"` is True; `not False` is True.
- **c) `False or not False and not True`: False** (1 pt) – `not False` is True, `not True` is False; `True and False` is False; `False or False` is False.
- **d) `"cat" > "Cat"` and `"10" < "2"`: True** (1 pt) – `'c' (99) > 'C' (67)` is True; `'1' (49) < '2' (50)` is True.
- **e) `len("neu") == 3 or (1 / 0 == 1)`: True** (1 pt) – Left operand is True; short-circuit skips division by zero.

Q13. Diagnose and Correct Common Decision Logic Errors (5 points)

Solution for Q13

Code / Situation	Problem / Logic Flaw	Correction / Valid Code
<pre>choice = input("Option: ") if choice == "A" or "B":</pre>	"B" is a non-empty string and thus always evaluates to truthy. The condition evaluates to True regardless of the user's input.	<pre>if choice == "A" or choice == "B": or if choice in ("A", "B"):</pre>
<pre>x = 0.1 + 0.2 if x == 0.3:</pre>	Floating-point binary roundoff error ($0.1 + 0.2 \neq 0.3$ exactly). Condition erroneously evaluates to False.	<pre>EPSILON = 1E-14 if abs(x - 0.3) < EPSILON:</pre>
<pre>val = input("Enter int: ") if int(val) > 0:</pre>	If the user enters non-numeric text ("abc"), int() crashes with a ValueError before the condition is tested.	<pre>if val.isdigit() and int(val) > 0: (short-circuit ensures safety)</pre>

Scoring: 1.5 pts for Row 1, 1.5 pts for Row 2, 2.0 pts for Row 3.

Q14. String Analysis Methods and Substring Inspection (5 points)

Solution for Q14

```
Prefix: True, Dash: True
Suffix Digits: True
Alphanumeric: False
```

Scoring Breakdown:

- **Line 1 (1.5 points):** code.startswith("NEU") is True; "-" in code is True.
- **Line 2 (1.5 points):** code[-4:] extracts "2026"; all characters are digits, so .isdigit() is True.
- **Line 3 & Explanation (2.0 points):** Alphanumeric: False (1 pt). Explanation (1 pt): The hyphen character '-' is neither a letter nor a numeric digit; thus .isalnum() evaluates to False.

Q15. De Morgan's Laws and Boundary Test Cases (5 points)

Solution for Q15

- **De Morgan's rewrite (2.5 points):**

```
temp < 18 or temp > 26
```

Full credit for exact boolean equivalence without outer not.

- **Critical boundary values (2.5 points):** The critical boundary test values are: **17, 18, 19** (testing lower boundary) and **25, 26, 27** (testing upper boundary). Full credit for stating the exact boundary edge values (18 and 26) along with their immediate adjacent values (17, 19, 25, 27).

Section C: Program Design & Logic Specification (25 points)

Model Solution for Section C (Tasks 1 & 2)

Task 1: Identify Decision Criteria, Inputs & Outputs (6 points)

- **Inputs & Types (2 pts):** weight (float), service (str), is_member (bool).
- **Validation Criteria (2 pts):** $0 < \text{weight} \leq 50.0$ and $\text{service} \in \{"S", "E"\}$.
- **Outputs (2 pts):** Base cost, express surcharge (if applicable), member discount (if applicable), and final payable amount.

Task 2: Structured Decision Logic in Ordered Steps (7 points)

Step 1: **Input Validation:** Check if $\text{weight} \leq 0$ or $\text{weight} > 50.0$ or $\text{service} \notin \{"S", "E"\}$. If true, print error and terminate.

Step 2: **Base Cost Calculation (Tiered Branching):**

- If $\text{weight} \leq 2.0$: $\text{base_cost} = 10.00$.
- Else if $\text{weight} \leq 10.0$: $\text{base_cost} = 10.00 + (\text{weight} - 2.0) \times 3.50$.
- Else: $\text{base_cost} = 38.00 + (\text{weight} - 10.0) \times 2.00$.

Step 3: **Surcharge Calculation:**

- If $\text{service} == "E"$: $\text{surcharge} = \max(0.40 \times \text{base_cost}, 8.00)$.
- Else: $\text{surcharge} = 0.00$.

Step 4: **Subtotal & Loyalty Discount:**

- $\text{subtotal} = \text{base_cost} + \text{surcharge}$.
- If $\text{is_member} == \text{True}$ and $\text{subtotal} > 30.00$: $\text{discount} = 5.00$; else $\text{discount} = 0.00$.

Step 5: **Final Cost:** $\text{total} = \text{subtotal} - \text{discount}$. Display formatted receipt.

Model Solution for Section C (Tasks 3 & 4)**Task 3: Hand Trace Calculation for $w = 6.0$, `service = "E"`, `is_member = True` (6 points)**

- Weight $w = 6.0 \in \text{Tier 2} \rightarrow \text{base_cost} = 10.00 + (6.0 - 2.0) \times 3.50 = 10.00 + 14.00 = \text{\$24.00}$ (2 pts).
- Express Surcharge: $40\% \times 24.00 = \$9.60$. Since $\$9.60 \geq \8.00 minimum, surcharge = **\\$9.60** (1.5 pts).
- Subtotal: $\$24.00 + \$9.60 = \$33.60$.
- Member Discount: Since `is_member = True` and $\$33.60 > \30.00 , discount = **\\$5.00** (1.5 pts).
- Final Payable Cost: $\$33.60 - \$5.00 = \text{\$28.60}$ (1 pt).

Task 4: Explain Design Decisions (6 points)**a) Why use `if-elif-else` rather than independent `if` statements? (3 points)**

Weight tiers are mutually exclusive. With independent `if` statements, a parcel weighing 1.5 kg would match multiple conditions if not carefully guarded, and all conditions would be evaluated redundantly. The `if-elif` construct guarantees that exactly one tier is selected and execution halts as soon as the correct branch is matched.

b) Why execute validation prior to calculations? (3 points)

Validating early (the "guard condition" pattern) prevents invalid or impossible operations (such as calculating negative weights or processing unrecognized service codes), prevents runtime errors, and saves computational overhead.

Section D: Applied Programming in Python (30 points)

Problem 1: Residential Electricity Billing with Progressive Tiered Tariff (15 points)

Reference Implementation for Problem 1

```
# Input
customer_name = input("Enter customer name: ")
prev_reading = float(input("Enter previous meter reading: "))
curr_reading = float(input("Enter current meter reading: "))

# Validation
if prev_reading < 0 or curr_reading < prev_reading:
    print("ERROR: Invalid meter readings. Readings must be non-negative and current >= previous.")
else:
    kwh = curr_reading - prev_reading

    # Tiered energy charge calculation
    if kwh <= 50:
        energy_charge = kwh * 0.12
    elif kwh <= 100:
        energy_charge = (50 * 0.12) + (kwh - 50) * 0.15
    elif kwh <= 200:
        energy_charge = (50 * 0.12) + (50 * 0.15) + (kwh - 100) * 0.20
    else:
        energy_charge = (50 * 0.12) + (50 * 0.15) + (100 * 0.20) + (kwh - 200) * 0.25

    # Excess consumption surcharge
    if kwh > 200:
        surcharge = 10.00
    else:
        surcharge = 0.00

    # Tax and Total
    subtotal = energy_charge + surcharge
    vat = subtotal * 0.08
    total_bill = subtotal + vat

    # Output
    print(f"Customer Name: {customer_name}")
    print(f"Electricity Consumed: {kwh:.1f} kWh")
    print(f"Energy Charge: ${energy_charge:.2f}")
    print(f"Excess Surcharge: ${surcharge:.2f}")
    print(f"VAT (8%): ${vat:.2f}")
    print(f"Total Bill: ${total_bill:.2f}")
```

Marking Rubric for Problem 1 (15 points)

- **Input & Data Types (2 pts):** Reads strings and correctly casts readings to float.
- **Input Validation (3 pts):** Correctly checks $\text{prev_reading} \geq 0$ and $\text{curr_reading} \geq \text{prev_reading}$, issuing error message if invalid.
- **Tiered Progressive Calculation (5 pts):** Correctly implements tiered bracket mathematics: Tier 1 (\$0.12, 1 pt), Tier 2 (first 50 at \$0.12 + remainder at \$0.15, 1.5 pts), Tier 3 (1.5 pts), Tier 4 (\$0.25 on remainder above 200, 1 pt).
- **Surcharge & VAT (2 pts):** Flat \$10.00 surcharge when $\text{kwh} > 200$; 8% VAT calculated on subtotal.
- **Formatted Output (3 pts):** Displays itemized receipt with exact formatting and `:.2f` currency values.

Problem 2: University Course Enrollment & Eligibility Verification (15 points)

Reference Implementation for Problem 2

```
# Constants
DIVIDER = "-" * 45
SUB_DIVIDER = "-" * 45

# Input
student_id = input("Enter Student ID: ").strip()
gpa = float(input("Enter Cumulative GPA: "))
credits_earned = int(input("Enter Completed Credits: "))
prereq_status = input("Prerequisite Status (pass/fail): ").strip()
warning_flag = input("Disciplinary Warning (yes/no): ").strip()

# Student ID Validation
if len(student_id) != 8 or not student_id.startswith("NEU") or not student_id[3:].isdigit():
    print("ERROR: Invalid Student ID format. Must be 'NEU' followed by 5 digits.")
else:
    # Eligibility Verification
    is_prereq_ok = (prereq_status.lower() == "pass")
    is_warning_free = (warning_flag.lower() == "no")
    is_credits_ok = (credits_earned >= 30)
    is_gpa_ok = (gpa >= 2.50)

    is_eligible = is_prereq_ok and is_warning_free and is_credits_ok and is_gpa_ok

    # Enrollment Classification
    if is_eligible:
        if gpa >= 3.60 and credits_earned >= 60:
            status = "APPROVED (Honors Priority)"
        else:
            status = "APPROVED (Standard)"
    else:
        # Determine rejection reason
        reasons = []
        if not is_prereq_ok:
            reasons.append("Prerequisite not passed")
        if not is_warning_free:
            reasons.append("Active disciplinary warning")
        if not is_credits_ok:
            reasons.append("Insufficient credits (< 30)")
        if not is_gpa_ok:
            reasons.append("GPA below 2.50 threshold")
        status = f"REJECTED ({', '.join(reasons)})"

# Formatted Output
print(DIVIDER)
print("ENROLLMENT ELIGIBILITY REPORT: DSAI2001")
print(SUB_DIVIDER)
print(f"Student ID: {student_id} | Cumulative GPA: {gpa:.2f}")
print(f"Completed Credits: {credits_earned}")
print(f"Enrollment Status: {status}")
print(DIVIDER)
```


Marking Rubric for Problem 2 (15 points)

- **Input Collection & Type Conversion (2 pts):** Prompts all 5 variables with appropriate type casting (float, int, str).
- **Student ID Format Validation (4 pts):** Correctly verifies length == 8, prefix "NEU" with .startswith(), and digits with .isdigit(); halts on failure.
- **Eligibility Decision Logic (4 pts):** Correctly evaluates the conjunction of all four criteria (prereq, warning, credits, gpa) using boolean operators.
- **Classification (Honors vs Standard vs Rejected) (3 pts):** Correct nested condition for Honors priority ($\text{gpa} \geq 3.60 \wedge \text{credits} \geq 60$); provides informative status.
- **Output Formatting (2 pts):** Formats report card with divider borders and properly aligned fields.